

Noteworthy 轻量化在线笔记软件

项目说明文档

俞烨 学号 102301116
选题方向：在线笔记软件

2026 年 6 月 17 日

目录

1	范围	2
1.1	标识	2
1.2	项目概述	2
1.3	运行说明	2
2	总体设计	2
2.1	技术选型	2
2.2	系统架构	2
2.3	数据模型	3
3	详细设计	4
3.1	功能规格	4
3.2	用户认证与授权	4
3.3	富文本编辑器	5
3.4	图片生命周期管理	5
3.5	全文搜索	5
4	技术难点与对策	5
5	程序结构与接口	6
6	总结与评估	7
6.1	学习收获	7
6.2	完成度评估	7
7	原创性声明与引用	7

1 范围

1.1 标识

项目名称为 Noteworthy，版本号 1.0，属于轻量化在线笔记软件类别。

1.2 项目概述

Noteworthy 是一个面向个人知识管理场景的在线笔记平台，提供从信息采集、组织到检索的完整笔记工作流。系统采用前后端分离架构，后端提供 RESTful API，前端为单页应用，通过 JWT 实现无状态认证。

该项目的核心特色包括：前后端完全自主开发，数据主权可控；基于 ProseMirror 内核的 Tiptap 编辑器提供专业级富文本编辑能力，支持 15 种以上扩展与代码语法高亮；采用 BCrypt 12 轮密码哈希配合 JWT 双令牌无感续期与用户级数据隔离，安全机制对标生产环境标准；双层图片清理策略配合归档笔记自动过期机制，有效解决服务器存储泄漏问题。

1.3 运行说明

运行环境要求 Docker 与 Docker Compose、Python 3.10 及以上版本、Node.js 18 及以上版本。

启动步骤如下：

1. 启动数据库，执行 `docker compose up -d db`
2. 启动后端服务：

```
cd backend && cp .env.example .env
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt && alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

访问 <http://localhost:8000/docs> 可查看 Swagger API 文档。

3. 启动前端服务：

```
cd frontend && npm install && npm run dev
```

浏览器打开 <http://localhost:5173> 即可使用。

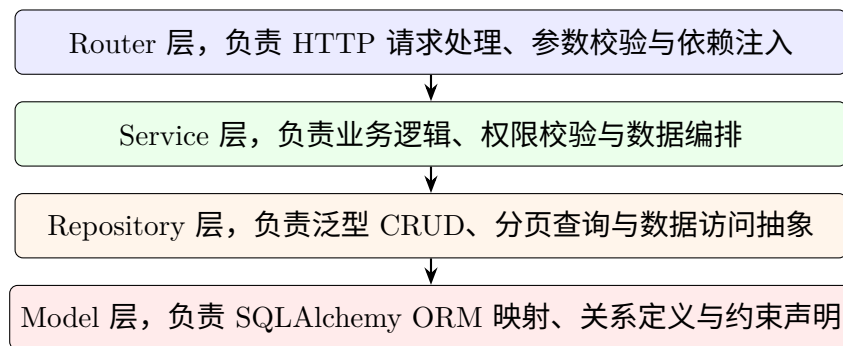
2 总体设计

2.1 技术选型

2.2 系统架构

后端采用四层分离架构，严格遵循单一职责原则，各层职责如下：

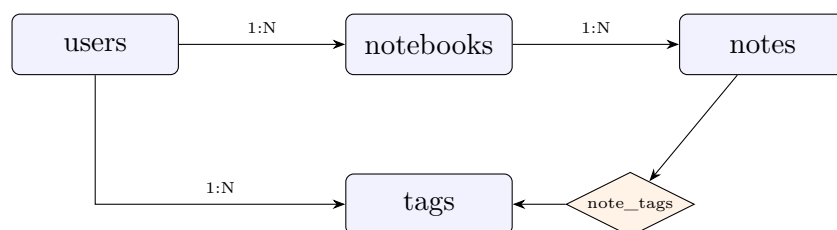
层面	技术	选型理由
前端框架	Vue 3 + TypeScript	采用 Composition API 与 script setup 语法，开发体验简洁高效，TypeScript 提供全量类型覆盖
构建工具	Vite 6	原生 ESM 热更新，冷启动低于 300 ms，较 Webpack 提升 10 至 100 倍
状态管理	Pinia	Vue 3 官方推荐方案，Setup Stores 与 Composition API 风格统一，TypeScript 类型推导完善
样式方案	UnoCSS	原子化按需生成，零运行时开销，兼容 Tailwind 生态
富文本	Tiptap	基于 ProseMirror 内核，可扩展架构支持表格、代码高亮、任务列表等企业级功能
后端框架	FastAPI	原生支持 async/await，自动生成 OpenAPI 文档，Pydantic v2 数据校验性能提升 5 至 50 倍
ORM	SQLAlchemy 2.0	声明式映射配合 select 风格查询与异步引擎，2.0 版本类型提示完备
数据库	PostgreSQL 16	JSONB 原生存储 Tiptap 文档，pg_trgm 三字符索引加速模糊搜索
认证	JWT 双令牌	Access 令牌有效期 30 分钟，Refresh 令牌有效期 7 天，无状态鉴权有利于水平扩展
容器化	Docker Compose	采用 postgres:16-alpine 镜像，healthcheck 就绪探针保障启动顺序



前端按功能域组织，API 模块封装接口调用与令牌刷新拦截器，Pinia Store 管理响应式状态，View 负责声明式渲染，UI 组件库提供一致的设计系统。

2.3 数据模型

系统共包含 5 张数据表，核心实体关系如下：



表名	关键字段	设计要点
users	id, username, email, hashed_password	采用 UUID 主键防止枚举攻击, username 与 email 建立唯一索引
notebooks	id, user_id, name, icon, color, sort_order	通过 CASCADE 级联删除笔记, sort_order 字段支持自定义排序
notes	id, notebook_id, user_id, content, plain_text, is_pinned, is_archived	content 字段使用 JSONB 类型存储 Tiptap 文档, plain_text 冗余字段用于加速搜索
tags	id, user_id, name, color	user_id 与 name 建立联合唯一约束以防止重复
note_tags	note_id, tag_id	多对多关联表, 采用双主键与 CASCADE 删除策略

3 详细设计

3.1 功能规格

功能域	能力描述
用户认证	支持注册与登录, 采用 JWT 双令牌鉴权机制, 实现令牌自动刷新与导航守卫
笔记本管理	支持创建、重命名与删除, 提供自定义图标与颜色配置, 支持拖拽排序与归档
笔记编辑	集成 Tiptap 富文本编辑器含 15 种以上扩展, 实现 3 秒防抖自动保存与 Ctrl+S 手动保存, 提供字数统计
笔记组织	支持置顶、归档与恢复, 实现标签多对多关联与跨笔记本聚合视图
图片系统	支持上传且限制 5MB 以内的 5 种图片格式, 提供即时孤立清理与定时全量扫描, 归档超过 7 天的笔记自动删除
全文搜索	基于标题与纯文本的 ILIKE 模糊匹配, 支持按笔记本过滤与分页
UI/UX	采用玻璃拟态设计风格, 支持深色模式与响应式三端适配, 提供骨架屏、Toast 通知与页面过渡动画

3.2 用户认证与授权

系统采用 JWT 双令牌认证机制。用户登录成功后, 后端签发 Access Token 与 Refresh Token, 均使用 HS256 算法签名, payload 包含 sub、iat、exp 与 type 四项声明。Access Token 有效期为 30 分钟, 用于接口鉴权; Refresh Token 有效期为 7 天, 用于无感续期。

鉴权流程中, 请求经由 get_current_user 依赖注入解析 Access Token, 校验 type 字段为 access 且未过期, 随后将用户对象注入路由函数。前端 Axios 响应拦截器在捕获 401 状态码后, 自动以 Refresh Token 调用 /auth/refresh 接口获取新令牌对。针对并发场景, 拦截器实现了请求队列机制, 首个 401 请求持有刷新权, 其余请求以 Promise 入队等待, 刷新完成后统一重试, 从而避免竞态导致 Token 失效。所有 Service 层查询均携带 user_id 条件, 确保用户间数据物理隔离。

3.3 富文本编辑器

编辑器基于 Tiptap 构建，其底层采用 ProseMirror 文档模型，集成了 15 种以上扩展：

- StarterKit，包含段落、标题、列表、引用与代码
- TextStyle 与 Color，提供 16 色预设与拾色器
- CodeBlockLowlight，实现 lowlight 语法高亮
- Highlight、Underline 与 Link
- Table 及 TableRow、TableCell、TableHeader
- Typography 与 Placeholder
- TaskList 与 TaskItem，支持嵌套
- CharacterCount，实现实时字数统计
- Image，采用服务端上传而非 Base64 内嵌
- 自定义 CodeBlockWithLabel，显示语言标签

保存策略方面，编辑器 onUpdate 事件触发 3 秒防抖定时器，标题变更同样触发保存流程；Ctrl+S 快捷键可立即保存；组件卸载时若 isDirty 状态为真则执行兜底保存。状态栏提供三态指示，分别为 Saved、Unsaved 与 Saving。

3.4 图片生命周期管理

图片上传通过 POST /uploads/images 接口完成，后端校验 Content-Type 为 JPEG、PNG、GIF、WebP 或 SVG 格式，且文件大小不超过 5 MB，随后以 UUID 重命名存储至 uploads 目录，返回 /uploads/filename 形式的 URL。

即时清理机制在笔记更新时对比新旧 content 中的图片引用集合，删除不再被引用的文件；笔记删除时同步清理关联图片。定时兜底机制由后台 cleanup_loop 异步任务每小时执行一次全量扫描，删除 uploads 目录中未被任何 note 引用的文件，同时删除 archived_at 字段超过 7 天的归档笔记及其标签关联。

3.5 全文搜索

笔记创建或更新时，后端递归遍历 Tiptap JSON 文档，提取 type 为 text 的节点，拼接为 plain_text 冗余字段。搜索接口基于 ILIKE 对 title 和 plain_text 字段进行模式匹配，支持按 notebook_id 过滤与分页。数据库初始化时已启用 pg_trgm 扩展，后续可平滑升级为三字符索引加速方案。

4 技术难点与对策

难点一为 JWT 并发刷新竞态问题。在单页应用中，多个异步请求同时收到 401 状态码时，若各自独立刷新令牌，后到的刷新请求会使先前的 Refresh Token 失效，导致其他请求永久失败。对策是在 Axios 拦截器中实现请求队列模式，设置 isRefreshing 互斥标志，首个 401 请求持有刷新权，其余请求以 Promise 入队等待；刷新成功后调用 processQueue 统一重试，失败则全部拒绝并执行登出操作，额外内存开销为常数级。

难点二为服务器图片存储泄漏问题。用户删除笔记或编辑移除图片后，文件系统可能残留无人引用的图片文件，长期运行将耗尽磁盘空间。对策是采用双层清理策略，即时层在 Note-Service 的 update 与 delete 方法中对比新旧 content 的图片集合，立即删除孤立文件；兜底层由 cleanup_loop 异步任务每小时全量扫描，比对 uploads 目录与所有 note content 中的引用，删除无主文件与归档超过 7 天的笔记。两层互补，确保零泄漏。

5 程序结构与接口

```
WebProgram/
+-- docker-compose.yml      # PostgreSQL 16 (alpine)
+-- docker/init.sql         # uuid-oss + pg_trgm
+-- backend/app/
| +-- main.py               # FastAPI app factory & lifespan
| +-- api/v1/               # Routers (auth, notebooks, notes, ...)
| +-- core/                 # config, security, database, exceptions
| +-- models/               # ORM (User, Notebook, Note, Tag, NoteTag)
| +-- schemas/              # Pydantic v2 schemas
| +-- services/             # Business logic + cleanup
| +-- repositories/         # Generic BaseRepository CRUD
+-- frontend/src/
| +-- api/                  # Axios client + refresh interceptor
| +-- components/{ui,layout} # Design system + layout
| +-- layouts/              # DefaultLayout, AuthLayout
| +-- router/               # Vue Router + authGuard
| +-- stores/               # Pinia (auth, notes, notebooks, ...)
| +-- views/                # 10 page views
+-- docs/                   # Project documentation
```

后端采用路由、服务、仓库、模型四层架构，前端按功能域组织。系统共暴露 16 个 API 端点：

方法	路径	说明
POST	/api/v1/auth/register	用户注册，返回双令牌
POST	/api/v1/auth/login	用户登录，BCrypt 校验
POST	/api/v1/auth/refresh	刷新令牌对
GET	/api/v1/auth/me	当前用户信息
GET/POST	/api/v1/notebooks	笔记本列表与创建
GET/PUT/DELETE	/api/v1/notebooks/{id}	笔记本增删改查
GET/POST	/api/v1/notebooks/{id}/notes	笔记列表与创建
GET/PUT/DELETE	/api/v1/notes/{id}	笔记增删改查
PATCH	/api/v1/notes/{id}/pin	置顶与取消置顶
PATCH	/api/v1/notes/{id}/archive	归档与恢复
POST/DELETE	/api/v1/notes/{id}/tags	附加与移除标签
GET/POST	/api/v1/tags	标签列表与创建
PUT/DELETE	/api/v1/tags/{id}	标签更新与删除
GET	/api/v1/search?q=kw	全文搜索
POST	/api/v1/uploads/images	图片上传

6 总结与评估

6.1 学习收获

通过本项目，掌握了前后端分离全栈开发的完整流程，涵盖 ER 设计、Alembic 迁移、RESTful API 开发与 SPA 交互实现。深入理解了 JWT 认证体系，包括双令牌策略、并发刷新队列、依赖注入鉴权与数据隔离机制。实践了 ProseMirror 文档模型，理解了 Tiptap 扩展机制、JSONB 存储策略与纯文本提取搜索优化方案。积累了异步编程经验，涉及 SQLAlchemy 2.0 异步会话、FastAPI 异步路由与 asyncio 后台任务。工程化能力方面，掌握了泛型 Repository 模式、双层资源清理策略与 Axios 拦截器设计方法。

6.2 完成度评估

全部 7 个功能模块均已实现，包括用户认证、笔记本增删改查、笔记增删改查、富文本编辑器、图片系统、搜索与标签以及 UI/UX。代码结构清晰，API 文档自动生成。

未来演进方向包括采用 tsvector 全文搜索配合 GIN 索引替代 ILIKE 以提升检索性能，引入 WebSocket 实现实时通知与多端同步，增加笔记导出为 Markdown 或 PDF 的功能，以及基于 CRDT 或 OT 算法实现协作编辑。

7 原创性声明与引用

本项目为个人独立完成的设计与开发作品，所有核心代码均为原创。

主要开源依赖如下：Vue 3，见 <https://vuejs.org/>；Pinia，见 <https://pinia.vuejs.org/>；FastAPI，见 <https://fastapi.tiangolo.com/>；SQLAlchemy，见 <https://www.sqlalchemy.org/>；Tiptap，见 <https://tiptap.dev/>；UnoCSS，见 <https://unocss.dev/>；Phosphor Icons，见 <https://phosphoricons.com/>；lowlight，见 <https://github.com/wooorm/lowlight>。